

Cuestionario Librerías de Entorno Gráfico

Tipos de librerías para entornos gráficos en Python

Maycol Slon Buitrago Manrique

Oscar Mahecha Camargo

Ingeniería de sistemas, Corporación Universitaria Remington

Compiladores

2026

Tabla de contenido

Preguntas del cuestionario	3
1 ¿Qué es una librería gráfica en Python y para qué se utiliza?	3
2 ¿Cuál es la principal diferencia entre Tkinter y PyQt en términos de funcionalidad y uso? ..	3
3 ¿Qué tipo de aplicaciones son más adecuadas para desarrollar con Kivy?	4
4 ¿Cómo se estructura una aplicación básica utilizando wxPython?.....	5
5 ¿Qué entornos de escritorio son compatibles con PyGTK?.....	6
6 ¿Cuál es la licencia bajo la cual se distribuye PySide y por qué es importante?	6
7 ¿En qué casos es preferible utilizar Pygame en lugar de otras librerías gráficas?	6
8 ¿Cómo puede Flask, junto con Flask-SocketIO, ser utilizado para crear aplicaciones gráficas interactivas?	7
9 ¿Qué características hacen de Dear PyGui una opción atractiva para desarrolladores que utilizan OpenGL?	8
10 ¿Cuál es el propósito principal de Matplotlib y cómo puede ser utilizado en el desarrollo de interfaces gráficas?.....	8
11 Bibliografía	9

Preguntas del cuestionario

1. ¿Qué es una librería gráfica en Python y para qué se utiliza?

Se puede definir una librería en python como un conjunto de módulos, clases y funciones que ya se encuentran programados y permiten construir interfaces gráficas, es decir, ventanas, botones, menús, cuadros de texto, gráficos o animaciones, todo ello sin que el programador tenga que estructurar cada componente desde cero. Estas librerías se encargan de comunicarse con el sistema operativo o con la tarjeta gráfica, de modo que quien desarrolla puede concentrarse en la lógica de su aplicación.

Son utilizadas para desarrollar aplicaciones de escritorio, videojuegos, herramientas de visualización de datos, aplicaciones táctiles y móviles, e incluso interfaces que se ejecutan en el navegador. Sabemos que Python cuenta con varias opciones, como Tkinter, PyQt, PySide, Kivy, wxPython, Pygame y Matplotlib. Cada una de ellas se adapta dependiendo del tipo de proyecto que se va a realizar.

2. ¿Cuál es la principal diferencia entre Tkinter y PyQt en términos de funcionalidad y uso?

Básicamente el factor diferencial es el alcance. Se sabe que Tkinter es la interfaz estándar de Python para poder maniobrar el conjunto de herramientas gráficas Tk que viene incluida con el lenguaje, porque decimos que viene incluida, porque Tk ya existía antes de python con su propio lenguaje que es Tcl, Tkinter hace de traductor entre el lenguaje Python y la estructura de Tk por lo que no requiere instalación adicional. También contiene los widgets esenciales, como etiquetas, botones, campos de texto, listas y lienzos. Realmente es muy práctico por la facilidad con que se puede aprender, aunque tiene sus limitaciones en cuanto a diseño y puede quedarse corto cuando las aplicaciones son más robustas.

Por otro lado, PyQt es un conjunto de enlaces de Python para el framework. Posee una colección mucho más amplia y variada de widgets, además incluye herramientas para multimedia, redes, bases de datos y diseño visual con Qt Designer, también y permite lograr interfaces que tengan una apariencia mucho más moderna y profesional. Una diferencia notoria con Tkinter es que debe instalarse aparte y probablemente su aprendizaje es un poco más complejo. En la siguiente tabla veremos de forma puntual las diferencias:

Aspecto	Tkinter	PyQt
Instalación	Ya viene incluida con Python	Requiere instalación aparte
Dificultad	Realmente es muy Sencilla, siendo ideal para aprender rapido	Es mas compleja y su curva de aprendizaje suele ser superior
Widgets	Muy básicos	Maneja mucha variedad y componentes avanzados
Apariencia	Plana, sencilla y tradicional	Mucho más moderna y personalizable
Herramientas	Un conjunto limitado, tiene pocas herramientas adicionales	Posee Qt Designer, multimedia, redes y bases de datos
Licencia	Esta incluido, es decir, licencia de Python	No es integrada, GPL o comercial
Uso ideal	Creación de aplicaciones pequeñas y prototipos	Creación de aplicaciones profesionales y complejas

3 ¿Qué tipo de aplicaciones son más adecuadas para desarrollar con Kivy?

Es un framework que está pensado para crear aplicaciones con interfaces táctiles. Es la herramienta de trabajo apropiada para la creación de aplicaciones móviles que funcionen en Android y iOS, sin dejar a un lado la aplicabilidad de el mismo código en entornos como Windows, macOS y Linux. Algo muy importante es que es de código abierto.

Con el también es posible desarrollar juegos sencillos, kioscos interactivos, creación de aplicaciones educativas y demás prototipos que requieren una interfaz moderna y que cuente con animaciones, puesto que dibuja sus propios widgets incluyendo aceleración gráfica mediante OpenGL ES, además cuenta con su propio lenguaje de diseño denominado llamado kv.

4 ¿Cómo se estructura una aplicación básica utilizando wxPython?

Este es un enlace de Python para la biblioteca wxWidgets, que es la que se encarga de utilizar los componentes nativos de cada sistema operativo. Para organizar una aplicación básica se puede hacer en cuatro pasos:

Principalmente creamos un objeto de la clase wx.App, este es el que representa la aplicación; luego vamos a definir una ventana principal con wx.Frame y posteriormente dentro de ella vamos a agregar los elementos que va a contener la interfaz, comúnmente se hace sobre un wx.Panel y organizados con sizers, por último, enlazamos los eventos con el método Bind(), estos eventos pueden ser por ejemplo un evento click, es decir, cuando el usuario da click a un botón. Debemos iniciar el ciclo de eventos con MainLoop(), para que pueda ejecutarse el aplicativo y mostrar la ventana emergente.

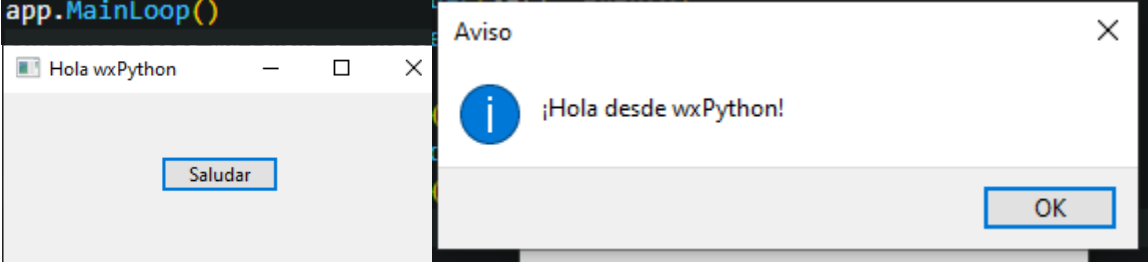
En el siguiente código se muestra una ventana con un botón que despliega un mensaje:

```
import wx

class Ventana(wx.Frame):
    def __init__(self):
        super().__init__(None, title="Hola wxPython", size=(300, 150))
        panel = wx.Panel(self)
        boton = wx.Button(panel, label="Saludar", pos=(100, 40))
        boton.Bind(wx.EVT_BUTTON, self.saludar)

    def saludar(self, evento):
        wx.MessageBox("¡Hola desde wxPython!", "Aviso")

app = wx.App()
Ventana().Show()
app.MainLoop()
```



5 ¿Qué entornos de escritorio son compatibles con PyGTK?

PyGTK es un enlace de Python para GTK 2. Esta es la biblioteca gráfica sobre la que se construyen muchos escritorios de Linux, es por ello que es compatible principalmente con los entornos de escritorio basados en GTK, estos pueden ser GNOME, Xfce, Cinnamon y MATE, es importante aclarar que también pueden ejecutarse en Windows y macOS, obviamente atendidos a una instalación adicional.

También hay que aclarar que PyGTK ya está obsoleto y solo funciona con Python 2, este fue reemplazado por PyGObject y ya nos da acceso a GTK 3 y GTK 4 desde Python, entonces los proyectos nuevos deben usar PyGObject.

6 ¿Cuál es la licencia bajo la cual se distribuye PySide y por qué es importante?

También hace parte de un conjunto de enlaces oficial de Qt para Python, conocido actualmente como Qt for Python, este se distribuye principalmente bajo la licencia LGPL, además también tiene las opciones GPL y comercial.

Que es lo llamativo de LGPL, que nos permite usar la librería en software propietario o comercial sin la necesidad de liberar el código fuente de la propia aplicación, siempre y cuando se cumplan las condiciones de la licencia, como, por ejemplo, enlazar la librería de forma dinámica y también permitir que el usuario pueda reemplazarla. En

Por otro lado, PyQt se ofrece bajo GPL o licencia comercial de pago y esto obliga a publicar el código fuente o tener que comprar la licencia.

7 ¿En qué casos es preferible utilizar Pygame en lugar de otras librerías gráficas?

Sabemos que Pygame es ese conjunto de módulos que están pensados para crear videojuegos y aplicaciones multimedia. Es viable implementarlo cuando el proyecto requiere un

ciclo de juego continuo, control directo sobre el propio dibujo en pantalla, el manejo de sprites, colisiones, animaciones, sonido, la lectura inmediata del teclado, el mouse o los controles.

Es por ello que es recomendado para juegos 2D, aunque también lo es para simulaciones y prototipos interactivos.

Para lo que si no es adecuada es para aplicaciones que requieran formularios o ventanas y necesiten muchos widgets, allí implementar librerías como Tkinter o PyQt resultan más prácticas y eficientes.

8 ¿Cómo puede Flask, junto con Flask-SocketIO, ser utilizado para crear aplicaciones gráficas interactivas?

Flask no es un framework muy robusto, por ello es denominado como el microframework web de Python, encargado de entregar la interfaz al navegador en HTML, CSS y JavaScript.

Este por sí solo trabaja con el esquema de petición y respuesta, es decir, la página cambiara solamente cuando el usuario la solicita.

En cambio, Flask-SocketIO si agrega comunicación bidireccional en tiempo real mediante WebSockets, es decir, el servidor puede enviar eventos al navegador en cualquier instante y el navegador también puede enviarlos al servidor, todo ello sin necesidad de recargar de nuevo la página. Es Gracias a esta maniobrabilidad que se pueden construir aplicaciones gráficas interactivas como paneles de monitoreo que se actualizan automáticamente, y muchas otras acciones como chats, tableros colaborativos, gráficos con datos en vivo y juegos en línea, y la ventaja es que pueden funcionar en cualquier dispositivo que posea un navegador.

9 ¿Qué características hacen de Dear PyGui una opción atractiva para desarrolladores que utilizan OpenGL?

Con Dear PyGui es posible crear interfaces gráficas de alto rendimiento, se basa en el enfoque de modo inmediato de Dear ImGui, que además incluye con renderizado acelerado por la GPU. Quienes trabajan con OpenGL, saben lo atractiva porque dibuja la interfaz directamente con la tarjeta gráfica, potenciando enormemente la fluidez incluso si tiene muchos elementos o datos que cambian constantemente, además usa las API gráficas disponibles en cada plataforma, como, por ejemplo, OpenGL en Linux.

También, incluye gráficas, tablas, editores de nodos y herramientas de dibujo. Es programable con pocas líneas de Python, siendo demasiado útil para herramientas de visualización de datos y aplicaciones técnicas que requieren funcionalidad en tiempo real.

10 ¿Cuál es el propósito principal de Matplotlib y cómo puede ser utilizado en el desarrollo de interfaces gráficas?

Es una de las librerías más famosas, nos permite crear visualizaciones estáticas, animadas e interactivas en Python.

Su objetivo principal es graficar datos, estos pueden demostrarse mediante líneas, barras, histogramas y diagramas de dispersión, siendo la base sobre la que pueden trabajar otras herramientas implementadas en el análisis de datos.

Para el desarrollo de interfaces gráficas Matplotlib puede incluirse dentro de ventanas de Tkinter, PyQt, PySide o wxPython por medio de sus backends, por ejemplo, FigureCanvasTkAgg, nos permite mostrar una figura como un widget más de la aplicación, siendo posible construir paneles donde sus gráficos puedan actualizarse según la interacción del usuario, al dar click en un botón o seleccionar cualquier dato.

11 Bibliografía

- Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media. <https://www.oreilly.com/library/view/-/9781491991725>
- McGugan, W. (2007). *Beginning game development with Python and Pygame: From novice to professional*. Apress. <https://doi.org/10.1007/978-1-4302-0325-4>
- Moore, A. D. (2021). *Python GUI programming with Tkinter* (2nd ed.). Packt Publishing. <https://www.oreilly.com/library/view/-/9781801815925>
- Rappin, N., & Dunn, R. (2006). *wxPython in action*. Manning Publications. <https://www.manning.com/rappin/>
- Ulloa, R. (2015). *Kivy – Interactive applications and games in Python* (2nd ed.). Packt Publishing. <https://www.oreilly.com/library/view/-/9781785286926/>
- Summerfield, M. (2008). *Rapid GUI programming with Python and Qt: The definitive guide to PyQt programming*. Prentice Hall.